culators—especially handheld calculators—were a very, very big deal.

So Allen's internship was working in the calculator group. He told me he'd told his managers all about me, that I was a great designer and had designed all these computers and things, and all of a sudden I found myself interviewing with a vice president of engineering, and the people under him, and the people under them. I guess they were impressed, because they made me an offer right away to come work there. They told me I could help design scientific calculators at HP. I thought, Oh my God.

I did love my job at Electroglas. I got to stand up all day, which I like, and help test and repair circuits. (A lot of their chips went bad because, instead of sockets, they used the soldered resistor-transistor logic [RTL] method of attaching chips.) I liked everyone I worked with and I'd made a lot of good friends. So when I told them about the job offer at HP, man, they did everything to keep me. They told me they'd make me a full engineer, they would up my salary over what HP had offered, and I felt bad because I really did love that company.

But even though Electroglas was what I considered to be a great job, it was nothing compared to what I considered to be the ideal job in the whole world: working on handheld scientific calculators at the only company in the world that could build a product like that. How could you beat that?

I was already a big fan of Hewlett-Packard. When I was at Berkeley, I'd even saved up to spend $400 (that's about $2,000 in today's money) on the HP 35.

There was no doubt in my mind that calculators were going to put slide rules out of business. (In fact, two years later you couldn't even buy a slide rule. It was extinct.) And now all of a sudden I'd gotten a job helping to design the next generation of these scientific calculators. It was like getting to be a part of history.

This was the company for me because, like I said, I'd already decided that I wanted to be an engineer for life. It was especially neat because I got to work on a product that at the time was the highlight product of the world—the scientific calculator. To me, it was the luckiest job I could have.

As an example of how great a company HP was, consider this. During this time—the early 1970s—the recession was going on and everyone was losing their jobs. Even HP had to cut back 10 percent on its expenses. But instead of laying people off, HP wound up cutting everyone's salary by 10 percent. That way, no one would be left without a job.

You know, my dad had always told me that your job is the most important thing you'll ever have and the worst thing to lose.

I still think that way. My thinking is that a company is like a family, a community, where we all take care of each other. I never agreed with the normal thinking, where a company is more competition driven, and the poorest, youngest or most recently hired workers are always the first to go.

By the way, I was twenty-two when I got that job at Hewlett-Packard.

• ○ •

Once I got into HP, I met a lot of people there and became good friends with the engineers, the technicians, even some of the marketing people. I loved the environment. It was just very free. I still had long hair and a beard, and no one seemed to mind. At HP, you were respected for your abilities. It didn't matter how you looked.

We had cubicles, I remember. For the first time, I sat in a cubicle and was free to walk around and talk to other people. During the day, you could throw out ideas about products and debate them. And HP made it easy to do that. Every day, at 10 a.m. and 2 p.m., they wheeled in donuts and coffee. That was so nice. And smart, because the reason they did it was so everyone would

More on HP

Stanford 1934 graduates Bill Hewlett and Dave Packard founded Hewlett-Packard in their garage in 1939. Now, a lot of people confuse that story with Apple's, saying that we started Apple in a garage. Not true. HP started in a garage, true. But in the case of Apple, I worked in my room at my apartment and Steve worked in his bedroom in his parents' house. We only did the very last part of assembly in his garage.

But that's how it goes with stories.

HP's first product was a precision audio oscillator, called the Model 200A. It generated sound waves and cost under $50, which was a quarter of the price of other companies' less reliable oscillators. And here's a cool fact. One of HP's earliest clients was Walt Disney Productions, which used eight Model 200B audio oscillators for testing the sound system for the movie *Fantasia*.

gather in a common place and be able to talk, socialize, and exchange ideas.

A few years before, during those long walks I took during high school, I'd decided that I was into truth and facts and solid calculations. I knew I never wanted to play social games. The Vietnam War only solidified that attitude. That's why I was sure, even at twenty-two, that I didn't want to switch from engineering to management, ever. I didn't want to go into management and have to fight political battles and take sides and step on people's toes and all that stuff.

I knew I could do that at HP—that is, have a long career without ever having to get into management. I knew this because I'd met a couple of engineers who were a lot older than me, and they had no desire to be in management either. So after I met them, I knew that was possible.

I worked at HP for quite a while—about four years. I didn't have a college degree yet, but I promised my managers I would work toward one by taking night classes at San Jose State nearby.

I couldn't imagine quitting my job and going back to school full-time, because what I was doing was too important.

• ○ •

At HP, I got into calculator circuits and how they were designed. I looked at the schematics of the engineers who had invented this calculator processor, and I was able to make modifications to those chips.

But the longer I worked there, the more I found myself drifting away from the computers of my past: computers and processors, registers, chips, gates, building all these things I used to be fascinated by. Everything was so good in my life; I just set my computer ambitions aside.

I'd even missed the fact that microprocessors—the brain of any modern computer today—were getting more and more powerful and more and more compact. I lost track of the chips that were coming up. I lost track of the fact that we were almost at the point where you could get all of a computer's main brainpower—its central processing unit (or CPU)—onto just one small chip.

I stopped following computer developments so closely. And I didn't really think of our calculators as computers, though of course they were. They did have a couple of chips inside that added up to a little microprocessor—a very strange one, I admit, but in those days you had to design things strangely and come up with weird techniques. Your chips could really do only one thing at a time. Back then, chips were simpler, you couldn't fit more than a thousand or so transistors on a chip, compared to more than a billion today.

So everything was weirder then. And because I was so happy in my job, I didn't know what I was missing.

• ○ •

What's a CPU?

You hear the letters "CPU" thrown around a lot, but what does it actually mean? And what did its invention change in terms of today's computer revolution?

CPU, short for central processing unit, is a term that's usually used interchangeably with "microprocessor." That is, provided the CPU is on one chip. When I first started building computers, like the Cream Soda Computer, there was no such thing as a CPU on a chip—that is, a microprocessor.

As it turned out, Intel came out with the first true microprocessor in the mid-1970s. It was called the 4004.

The whole purpose of the CPU, which really is the brains of a computer, is to seek and execute all the instructions someone stored in the computer as a program. Say you write a program that spell-checks a document. Well, the CPU is capable of finding that program (which is represented in the machine as the binary numbers 1 and 0) and communicating with the other components of the computer to make it run.

Sometimes a bunch of us engineers would take small planes and fly to lunch somewhere. A lot of us had our pilot's licenses. For my first flight ever in a small plane, I ended up in Myron Tuttle's plane. Myron was a design engineer, like me, a guy who worked with me in my cubicle. That day he let me sit in the copilot's seat, which I thought was so cool.

I remember there were two people in the back, other people in our group. So here we are, flying for lunch to Rio Vista, near Sacramento.

When Myron landed, we just bounced and bounced and bounced. I had never been in such a small plane before, so I just thought, Oh, this is interesting. So this is how a small plane is. Really bumpy when you land.

At lunch, the other pilots had this private conversation. (I found out later they were trying to decide whether they would let Myron fly us back!) Well, it turned out they decided, okay, it was just one flight, and the runway in San Jose was 10,000 to 12,000 feet long. They thought maybe Myron would be able to do a better job on the return flight.

So we flew back after lunch—and there it was again, another one of those really, really bouncy landings. Again I just thought that's how you land in small planes. There was a first bounce, then a second bounce that was pretty hard, then a scraping sound, and then it bounced, bounced, bounced, and bounced again for what seemed like the millionth time down the runway.

I must've been white as a sheet, I think everyone was. And not one of us could say a word. We taxied around the runway for a few minutes, and still the three of us didn't say a word to Myron. Not one word.

That silence was uncomfortable. Finally I felt like I had to say something, just anything technical, because he's an engineer and all. So after we got out of the plane, I said to Myron, "Hey, that's interesting that they bend the propeller like that—is that for aerodynamic reasons?"

And Myron said, "They don't." That's all he said.

I realized I had just said the worst possible thing.

Myron had bent his propeller on that landing.

To be fair to Myron, it's not impossible that I did something in my copilot seat that made the bounce worse. It's possible that in my own fright I touched something I shouldn't have.

At any rate, I heard Myron never flew again after that. As for the propeller he bent, he had to buy it. We mounted it on the lab wall, something for us to always look at and remember. Like it was a joke.

• ○ •

I think most people with day jobs like to do something totally different when they get home. Some people like to come home and watch TV. But my thing was electronics projects. It was my passion and it was my pastime.

Working on projects was something I did on my own time to reward myself, even though I wasn't getting rewarded on the outside, with money or other visible signs of success.

One such project I called Dial-a-Joke. I started it about two weeks before I went to work at HP, and it went on for a couple of years after that.

Now, a lot of people start companies, and I know a lot of people will probably be reading this book only because I started Apple. But what I wish more people knew about me is what I think I should really be famous for: creating the very first Dial-a-Joke in the Bay Area, which was one of the first in the world.

A dial-a-joke service was something I had wanted to do for a while, mostly because I'd been calling dial-a-joke numbers (remember Happy Ben?) all around the world with my Blue Box. So I knew there were dial-a-joke lines in places like Sydney, Australia, and Los Angeles, but there were none in the San Francisco Bay Area. How could that be? I couldn't believe it. And you know me; I always like to be in the forefront of things. So I decided I was going to be the first one to do it.

Before long I really did have the first dial-a-joke in the Bay Area, and it was unbelievably popular. In fact, it had so many calls that I could only keep doing it for a couple of years. I was fielding thousands of calls a day by the end of it. Eventually I couldn't afford it anymore.

• ○ •

To do a dial-a-joke system, the first thing was to get an answering machine. You couldn't just buy them. It was illegal to connect one to your phone line without actually renting it from a phone

company. Keep in mind that there were no phone jacks in the walls back then. Just wires connected to screws.

I knew movie theaters had answering machines, though. That was for prerecording movie titles and showtimes. Somehow I managed to rent one of those machines for about $50 a month. That was pretty expensive for a young guy like me. But I wanted to do it for fun, and money wasn't going to stop me. Well, at least not at first.

Next, I needed jokes. I got them from *The Official Polish-Italian Joke Book*, by Larry Wilde. That book was the best-selling joke book of all time.

So I hooked up the machine and recorded a joke. Using my best Slavic accent, I'd say: "Allo. Tenk you fur dialing Dial-a-Joke." Then: "Today's joke ees: Ven did a Polack die drinking milk? Ven de cow sat down! Ah, ah. Tenk you fer dialin' Dial-a-Joke."

The first day, I just gave the number to a few people at work and told them to let their kids try it.

The next day, I read another joke into the machine. And every day I'd do that, reading a new Polish joke into the machine.

You wouldn't believe how fast Dial-a-Joke ramped up. The first day, there were just a couple of calls. Then there were ten. The next day, there were maybe fifteen. And then suddenly it spiked up to a hundred calls, then two hundred calls a day. Within two weeks, the line was busy all day. I would call it from work and I couldn't even get through. After school let out that year, there were like two thousand calls a day on a single line phone number. I made a point of keeping my jokes as short as I could—under fifteen seconds—just so I could handle more calls a day. I couldn't believe how popular it got!

I started to really have a blast with it. Every now and then, just for fun, I'd take live calls when I got home from work. I'd say, "Allo. Tenk you fur dialing Dial-a-Joke." I got to talk to lots of

people and hear weird things about their schools and teachers and other students. I took notes. That way, if I asked someone (in my Polish accent, of course) what high school do you go to, and they answered "Oak Grove," I could say, "Hey, does Mr. Wilson still wear those weird red pants?"

So they were amazed by me. They heard the recordings and they knew I actually picked up the phone sometimes—they thought this old Polish guy knew everything about them! I told them my name was Stanley Zebrazutsknitski.

At one point I bought two books of insults—*2,001 Insults*, volumes 1 and 2. A lot of these insults were really funny. Sometimes I would say something a little critical to a caller—like, "You not so bright, are you?"—just to get them going. Usually they would retort by calling me something nasty, like an old fart. That's when I could start reading the insults out of the book, ones that were so clever no one could come back with anything good. As hard as anyone tried, I would always win the insult battle.

Somewhere around that time, I got complaints from the Polish American Congress that the jokes defamed people of Polish descent. Being a Polish Wozniak who tells and laughs at Polish jokes, I asked them if they would mind if I switched to Italian jokes. They said that would be fine.

See, the notion of political correctness didn't exist back then. The Polish-Americans didn't care if I told ethnic jokes as long as they weren't about Polish people!

Want to Hear a Dial-a-Joke?

The first dial-a-joke service is rumored to have been created by New York Bell in the early 1970s. Want to hear some examples? You can hear archived recordings at http://www.dialajoke.com.

And you know what? Twelve years later the same Polish American Congress gave me its Heritage Award, its highest award for achievements by a Polish-American.

• o •

As it happened, most of my callers were young teenagers. Adults don't have the time or the patience to keep dialing a busy number over and over to get through.

But the kids, because they were dialing it over and over, frequently misdialed the number. One time, on a weekend, I took a live call from this woman who said, "Please, you've got to stop that machine. My husband works nights and he's got to sleep days, and we're getting a hundred calls a day that are meant for you." So the next day I called the phone company and had them change the number. I did that just for her.

I didn't hear any more complaints for the next month, so I assumed the phone number switch worked. But a manager at the phone company called me to tell me that a lot of other people were complaining.

And that was frustrating to me because I didn't want to make trouble for anyone. So I started thinking about getting an easy-to-dial number. I was in Cupertino where one of the prefixes was 255, so I thought, How about 255-5555? That would be easy to dial—you could keep dialing the same touch-tone, and your finger wouldn't have to leave the space. I tried calling this number, and I found out that no one had it. I also found out that nobody had 255-6666.

I called a manager at the phone company—Dial-a-Joke was such a big deal by now that even shy Steve Wozniak could talk to phone company managers. I suggested that the remedy for all the misdialing should be an easy-to-dial phone number. I asked first for the 255-5555 number, but they weren't allocating numbers in the 5000 range. So I said, "How about 255-6666?" He checked and said, "Fine." And he gave it to me.

I ended up getting some cards printed up that said: "The Crazy Polack. Heard a good one lately? Call 255-6666."

I figured that would be the end of the misdialing problems, but it wasn't. I remember coming home from Hewlett-Packard to the apartments in Cupertino, where I lived, and there were three people waiting. They said they worked at Any Mountain, which was and still is a major ski supply shop in California. And their number was 255-6667, one digit different. They said they were getting so many crank calls and weird people and kids calling they were afraid to answer their own phone! I was kind of proud of the fact that my little operation was able to affect that big a business, but I really did want to change my number again to protect them. So I did that. I changed it to a 575 prefix—575-1625—but that 575 prefix was actually set up for high-volume calls like radio station contests and that kind of thing. And I had that number until the end of Dial-a-Joke a couple of years later.

But Dial-a-Joke was hurting for money. The cost of the answering machine alone was breaking me.

At one point I thought maybe I could get money from the callers to help pay for Dial-a-Joke. I added the message, "Please send money to P.O. Box 67 in Cupertino, California." In three months I received only $11. Only once did I get a whole dollar. Usually I'd get a nickel, dime, or quarter taped to a piece of paper.

• ○ •

The biggest problem with Dial-a-Joke, like I said, was the expense. Not only did renting the machine cost a lot of money, but I was constantly having to rent new machines from the phone company.

To give you an idea, in theaters, these machines lasted years. But with me, they were lasting, like, a month. So every month I'd have to call up the phone company and say, "You've got to come over here to fix your answering machine, it's no good."

And really I loved doing that because they were charging me so much to rent it, it seemed only right that I wouldn't have to be stuck with it once it broke down. I liked to see them lose money, too. So this guy would show up at 5 p.m., when I got home from work, with a whole new machine. I'd meet the guy, let him into the apartment, he'd install the machine, and that was that.

One month, when I got home that day after five when the repairman was supposed to be there, there was instead a note from him saying he'd been there at 2 p.m.

Two p.m.? I called up the phone company. "He's always supposed to come after five. You better have him come after five tomorrow." Well, the next day I got a note saying he'd been there at 3 p.m. So now I called the phone company almost livid—and that is really unusual for me—and I said something like: "You'd better tell him to be there at 5 p.m. this time." But then the next day, again, there was a note saying he'd been there at 2 p.m. What was going on? I had no idea.

But I had gone three days with a nonworking machine that I was paying for, and that was no joke to me.

Now, I decided to play the game a different way. I called them and this time just very politely asked them to get the guy there at five. I hooked up an illegal but working answering machine to my Dial-a-Joke phone and left a message in my Slavic voice that told all the kids the machine was broken because of the phone company, and if they liked Dial-a-Joke they better call 611 (the number for telephone repair) to complain. And I told them to have all their friends call, too.

The next day I was pretty much in meetings all day at Hewlett-Packard, but I got home at 4:45 p.m., just in time to disconnect the illegal answering machine before the telephone guy got there. Then I called 611 and said, "I have a complaint."

She said, "I know. Dial-a-Joke."

"How did you know?" I asked.

A Good Number Is Hard to Find

I told you about 255-6666. That was the first good phone number of my life. Many years later, I got the home number 996-9999, which had six digits the same. That was a milestone for me. When I lived in Los Gatos, I got numbers like 353-3333 and 354-4444 and 356-6666 and 358-8888.

My main goal with phone numbers was to someday get a number with all seven digits the same. The way they divided phone numbers between San Jose and San Francisco, all of those numbers went to San Francisco. For example, 777-7777 was the *San Francisco Examiner*. But as the area codes started running out of phone numbers, they started duplicating the prefixes, allowing San Jose's area code to someday have numbers that started with 222, 333, 444, or whatever.

In the early days of cell phones, I had a scanner that would let you listen to people's cell phone calls. It would show me the phone numbers of callers. One day my friend Dan spotted a number in our 408 area code starting with 999. I immediately called the phone company to get 999-9999 for myself. Unfortunately, they couldn't pull that number out of a larger group of numbers someone else had reserved.

A few weeks later, Dan spotted a number starting with 888. This time I lucked out.

I got the numbers 888-8800, 888-8801, up to 888-8899. So by about 1992, I had achieved my lifetime goal of having the ultimate phone number.

I put the number 888-8888 on my own cell phone, but something went wrong. I would get a hundred calls a day with no one on the line, not once. Sometimes I would hear shuffling sounds in the background. I would yell, whistle, but I could never get anyone to speak to me.

Very often I would hear a tone being repeated over and over, and then it hit me. It was a baby, pressing the 8 button over and

over. I did a calculation that concluded that perhaps one-third of the babies born in the San Jose 408 area code would eventually call my number. And basically this made my phone unusable.

I'll tell you about one last number. It was 221-1111. This number has a mathematical purity like no other. It's all binary numbers—magic computer numbers. Powers of two. But the real purity was how small the digits were, 1s and 2s. By the rules of allocating phone numbers in the United States, no other phone number could have only two 2s and the rest 1s. In that sense, it was the lowest number you could get.

It was also the shortest dialing distance for your finger to move on a rotary phone,

As with 888-8888, I got so many wrong phone numbers every day. One day I was booking a flight and noticed that Pan American Airlines had the number, 800-221-1111.

The next phone call I got, I heard someone start to hang up after I said hello. I shouted, "Are you calling Pan Am?" And a woman came on the line and said, "Yes." I asked her what she wanted and booked my first flight for a Pan Am passenger that day.

Over the next two weeks, I booked dozens of flights. I made up a game to see how crazy I could make prices and flight times and still have people book it. After a couple of weeks, I started feeling guilty. And vulnerable. I didn't want to get arrested. So for the next two years, I answered every phone call with, "Pan Am, International Desk. Greg speaking." My friends would have to yell, "Hey, Steve, it's me," when they called. I would trick people into booking the craziest things, but I would always tell them it was a prank and that I was not really Pan Am.

For example, I might tell them that their flight would leave San Jose at 3 a.m., so a lot of times they would be really relieved. I started booking callers on what I called the "Grasshopper Special." If they flew through our lesser-used airports, it would reduce their fare. I almost always told them to fly to Billings, Montana, down to Amarillo, Texas, then up to

Moscow, Idaho, then to Lexington, Kentucky, and *then* to their destination. Boston.

Hundreds of people took me up on this. Hundreds, maybe thousands, over the course of two years. Anyone who knows me saw me taking reservations constantly. I also booked Grasshopper flights to other countries, telling people they had to stop in Hong Kong, Bangkok, Tokyo, and Singapore to get to Sydney.

I told some callers they could fly "freight." But they had to wear warm clothing.

I kept a straight face because everyone always went for the lower fare. At some point I started telling them it was cheaper to fly on propeller planes than jets. The first time I did this, I tried to book a guy on a thirty-hour flight to London. But he would have nothing to do with it. I did get a number of people to buy into a cheap twenty-hour flight from San Jose to New York City.

The craziest one—and I still smile when I think about it—was the one I called the "Gambler's Special." I would tell them that the first leg of their flight had to go to Las Vegas. From there, they had to go to our counter at the airport. And if they rolled a "7," the next leg would be free.

"Every other call today has been for Dial-a-Joke," she said, sounding really frustrated. So I just got this big grin on my face. I felt like I had made the big time.

And yes, the guy did show up that day at 5 p.m.—with his supervisor. I let the guy in to replace it, but left the supervisor out in the rain with a book to read called *I'm Sorry, the Monopoly You Have Reached Is Not in Service*, by K. Aubrey Stone. It's a really lousy book, actually, but I thought he deserved it.

Eventually I had to give up Dial-a-Joke because I couldn't keep it up on my tiny HP engineer's salary. Even though I loved it so, so much.

• ○ •

There is one major thing I haven't yet told you about Dial-a-Joke. It is how I met my first wife, Alice. She was a caller one day when I happened to be taking live calls. I heard a girl's voice, and I don't know why but I said: "I bet I can hang up faster than you!" And then I hung up. She called back, I started talking to her in a normal voice, and before long we were dating. She was really young, just nineteen at the time.

We met, and the more I talked to her, the more I liked her. And she was a girl. I had only kissed two girls up to that point, so even being able to talk to a girl was really rare.

Alice and I were married two years later. And our marriage lasted just a little longer than my career at Hewlett-Packard, which is funny in a sad way.

Because I thought both of those arrangements were going to last forever.

Chapter 9

Wild Projects

During those four years at HP, from the time I was twenty-two to twenty-six, I constantly built my own electronics projects on the side. And that's not even including Dial-a-Joke. Some of these were really amazing.

When I look back, I see that all these projects, plus the science projects I did as a kid and all the stuff my dad taught me, were actually threads of knowledge that converged in my design of the first and second Apple computers.

After Dial-a-Joke, I was still dating Alice, still living in my first apartment in Cupertino, still coming home every night to watch *Star Trek* on TV and work on my projects. And there was almost always some kind of a project to work on, because after a while, people at HP started talking about my design skills to their friends, and I started to get calls from them. Like, could I go down to some guy's house and design something electronic for him? Gadgets, stuff like that. I would always do it for no money—I'd say, Just fly me down to Los Angeles and I'll bring the design down and I'll get it working. I never charged any money for it because this was my thing in life—designing stuff—this is what I loved to do. As I said before, it was my passion.

My boss, Stan Mintz, once came to me with a project to do a

home pinball game. These friends of his wanted to build a little pinball game with rockers and buttons and flippers, just like the ones you use in arcades. So I basically designed something digital that could watch the system, track signals, display the score, sound buzzers and all that. But there was one very tricky circuit that confused Stan, and I remember him telling me, "No, that's wrong. It won't work." But I showed him why it would work. And it did.

I just loved it whenever other engineers, especially my boss, would be surprised by my designs. That always made me happy.

• ○ •

And soon I was getting involved in one of the most amazing projects. Someone asked me to help design the digital part of the first hotel movie system, which was based on the very earliest VCRs. No one had VCRs then, of course. I was thinking, Oh my god! This is going to be incredible—designing movies for hotels! I couldn't get over it.

Their formula was this. They'd line up about six VCRs. Then they had a method of sending special TV channels to everybody's room. They could play the movies on those channels. There was a filter in each room to block those channels. But the hotel clerk in the lobby could send a signal to unlock the filter in a particular room. Then the guest could watch the movie they ordered on their TV. Someone in the VCR room had to literally start the movie, but this was still a really cool system.

Another project I did was for a company that came out with the first consumer VCRs, and yes, it was before the Betamax. It was called Cartrivision, and the VCR had this amazing motor in it with its own circuit board that spun as it ran the motor. In other words, the spinning circuit board was actually the electronics that ran the motor! It was very strange.

Well, at HP I heard a rumor that this little company had gone bankrupt and they had about eight thousand color VCRs for sale, cheap. I mean, at the time a black-and-white VCR for a school

cost almost $1,000. But Cartrivision was selling them at this super-low surplus price. So my friends and I would drive down to their San Jose tape duplication facility. And we'd walk through the building, just amazed at these hundreds of color VCRs in boxes. They weren't really cabinet VCRs like you've seen, but kind of open, where you could see all the circuitry. Anyway, we'd take a bunch of engineers down there and buy them for, like, $60.

This became a huge part of my life almost right away. I studied the kinds of circuits the VCR used, how it worked, went through all the manuals. I tried to figure out how they processed color, how color got recorded onto tape, how the power supply worked. This was all information that came in really handy when we did the color Apple computers. And then I would buy wooden boxes to put these naked color VCRs into. Listen, I had a working color VCR in my apartment in Cupertino when nobody, but nobody, in the world had a VCR at home.

There were only a few movies available at the time. The first one I watched at home was *The Producers*. I saw it right there on my Cartrivision. I opened up my TV, looked at the schematic to figure out where the video signal was, and figured out how to match it to the Cartrivision. That way, I could record shows, too. One of the things I actually recorded was Nixon's resignation on TV. So I must be one of the only people in the world to have a consumer tape of that, because if you go back to that date in time, 1974, you'll see that there were absolutely no VCRs on the market available to consumers.

• ○ •

Now let me tell you about Pong. Remember Pong? It was the first successful video game (first in the arcade, later at home), and it came from a company called Atari. I remember I was at the Homestead Lanes bowling alley in Sunnyvale with Alice, who was by now my fiancée. And there it was, Pong. I was just mesmerized.

Pong really stood out for me because it was a full-size arcade game right there in a bowling alley. Back then, bowling alleys had a bunch of pinball machines everywhere, but never, ever anything electronic. And Pong was so different from those. It had this little black-and-white TV screen with digital sounds coming out of it—pong, pong, pong. You used the dials to move a paddle up and down to hit a little white ball and bat it to the other player's paddle. It was so simple, but so fun.

All I could do was stare at it in amazement. And I noticed that while pinball games cost a dime to play and required only one person to play, this game cost a quarter and needed two people.

The thing I thought was so incredible wasn't so much the game concept—I mean, it was very much like Ping-Pong or tennis or something like that—as the fact that somebody had come up with the idea that by controlling the white and black dots (pixels) on a TV screen, you could actually build a game. Wow!

And it was a game so different from pinball, but still very attractive. In fact, I found it even more attractive, because of its newness, than all those flashy pinball machines. I got some quarters and played a few games with Alice, and then I sort of stood there awhile and stared at it. Alice said, "What? What are you thinking?"

"What? Here's what," I said. "I could design one of these."

I knew the minute I started thinking about it that I could design it because I knew how digital logic could create signals at the right times. And I knew how television worked on this principle. I knew all this from high school working at Sylvania, from the hotel movie system, from Cartrivision, from all kinds of experience I'd already had.

So right there in that bowling alley I suddenly had this cool new goal. I was going to go back and start thinking about my first design that was actually going to put characters on a TV set. I remember how, way back in high school, I wondered how, if I

ever did a computer, I would ever be able to afford one that could display characters on a screen. That was unfathomable back then. But now, I knew, something was different.

Everything had changed.

• ○ •

I right away decided I was going to build my own Pong game, for my own use at home, and that meant I had to design it from scratch.

To understand how I did this, you have to know a little bit about how a TV set works. It draws a regular pattern, in little dots, in lines across the screen. Left to right for the top line, left to right for the next one, left to right for the next one down, and so on. When it gets done with all 575 lines, it starts again. There's a precise interval, too, between the drawing of each line. All of this is part of what's known as the National Television Systems Committee (NTSC) standard, which is the standard all TVs in the United States follow.

So I understood exactly what the right timings were. I figured out exactly how I could use chips to delay the amount of time the lines scanned on the TV and generated a dot on the screen at the right moment. I also kept track of where it was drawing dots at any point in time.

So if you look at an NTSC television set, there might be 300,000 total possible dot positions, each corresponding to where the line is at any point in time. And remember, each one of these dot positions is getting hit as the TV draws the picture line by line, left to right, from top to bottom, really fast. It happens about 60 times a second. I figured out that I should be able to design a circuit that could keep up with the timing and generate TV signals to draw dots in other places on the screen.

One of my skills was that I was really good at designing things with the absolute minimum amount of chips. That goes back to the Cream Soda Computer. So I figured out how to put just a few

chips together and use a crystal clock chip (like the one in my Blue Box or the one that keeps time in your watch) to control the timing and keep count of what is happening.

TVs at that time didn't have any video input connections. There was no video-in like there is now. And I needed a video-in connection if I was going to design a game that would let you dis-play the game on-screen. But how could I figure out where on the TV the video came in from the antennas?

Well, all TVs came with schematics back then. And if you read the schematics and you knew electronics, you could study the transistors and the filters and the coils and the voltages. You could trace your way through the circuit and find out where the video signal really exists in the TV.

So that's where the signal goes into the display circuits of the tele-vision set, the signal that carries the television picture according to NTSC standards. I tapped around with an oscilloscope, and with a few resistors and test points I was able to find the exact point of the video signal inside the TV. So I just applied my own video signal to that point, and from then on I could generate everything on the screen.

I also could've put my own TV signal on a TV channel through what is called a modulator. It's the same way a VCR, for instance, puts a TV picture on Channel 3. But my other method was more efficient—better and easier—for me at the time.

So this Pong game I did, it wasn't commercial, of course. I did it all on my own, at home. It had nothing to do with Atari, but I did do it at least a year before Atari came up with a home Pong game that worked with your TV.

All in all, I ended up with twenty-eight chips for the Pong design. This was amazing, back in the days before microproces-sors. Every bit of the game had to be implemented in wires and small gates—in hardware, in other words. There was no game program—that is, a game in software form that someone could load. It was all hardwired.

Well, I wanted to make mine even more special, so in addition to showing the score on-screen, I programmed these little chips (called PROMs, for programmable read-only memory) to spell out four-letter words every time you missed the ball. You know. Like HECK or DARN. Not exactly those words, but this is a family book. Anyway, I could easily turn the four-letter-word feature on or off with a switch.

Once, while visiting Steve Jobs, who was working at Atari, I showed it to a bunch of engineers there and they loved it. Soon after, I showed it to Al Alcorn, who was one of the top guys (next to founder Nolan Bushnell) at Atari, and he was really impressed! They thought it was funny, with the dirty words and all.

They offered me a job right then and there, but I said, No way. I explained to them that I could never leave Hewlett-Packard. It wasn't possible. My plan, I told them, was to work at HP for life. It was the best company for an engineer like me.

• ○ •

A few months later—and of course I was still at Hewlett-Packard—I got a call from my friend Steve Jobs. He was getting excited about some interesting work he was doing at Atari. Atari was getting all kinds of attention by then for having started the video game revolution with games like Pong. Its chief at the time, Bushnell, well, he was just larger than life. Steve said it was a blast to work for him.

So anyway, Steve had this job at Atari. After the people designed the games at Atari's Grass Valley design facility, they'd send them down to Steve in Los Gatos. And he would look at those games and try to give them, you know, some final tweaks. Whatever could make them just a little bit better, he would do. Or he might find bugs.

Steve called me at work one day saying that Nolan wanted to do another Pong-like game. Nolan wanted me to do it because he knew how good I was at doing designs with the fewest possible

chips. Nolan had been complaining that the Atari games were going higher and higher in chip count, approaching two hundred chips for a single game. He wanted them to be simpler. And he'd seen how good I was at that.

Steve said Nolan wanted a one-player version of Pong, but with bricks that would bounce the ball back to the paddle.

"You gotta get in here," he said. "They're right. You'd be perfect for it."

I was immediately excited because I could see that if one player could play it, instead of it needing two players, it could be a much more fun game. Because when the ball breaks enough bricks—do you remember this game?—it can then get behind the bricks and start bouncing them from behind, which bounces even more bricks out. So it's a little more complicated, and you don't need someone else to play.

So, not even thinking about it, I said, "Sure."

And then Steve says, "Well, there's a caveat. It has to be done in four days." Wow! Back then there wasn't a game you could do in four days. Plus, a game was all hardware. It was hardware where every single wire mattered and every single connection had to determine when signals would be on the screen. And then, I mean, there were the thousands of little connections between chips, and they all mattered, and I realized that this timetable was ridiculous. A game like that should take engineers working on a normal schedule a few months to complete.

I realized I could probably do this in a shorter time than anyone else, but I still thought it was an insane goal to do a hardware game in four days.

But I was up for the challenge.

• ○ •

So I designed this game Breakout.

I began by actually drawing the schematics so a TV would display light on the screen—line by line. I didn't sleep for four days

and nights during this project. During the day, I drew the design on paper, drawing it out clearly enough so that a technician could take the design and wire chips together. At night Steve would wire the chips together, using a technique called wire-wrapping. Wire-wrapping is a way of connecting chips with wires that does not require soldering. I prefer soldering, myself, because it's always cleaner, smaller, and tighter. But wire-wrapping is how most technicians do it. Don't ask me why.

With wire-wrapping, you hear a zipping sound of a little electronic motor, the sound of it wrapping a wire around a small metal pole. In about a second the wire-wrap gun wraps the wire about ten times around the metal pole. Then zip onto another one. Zip onto another one. Over and over. It actually gets kind of messy, with wires dangling everywhere between the metal poles. But like I said, that's how things are done—a lot of engineers still do it that way. I can't understand why, but they do.

Well, then Steve would breadboard it—that means putting all the components, wires, chips, and everything, onto a prototype board—and do the wire-wrapping.

It's funny how when you're up so late at night for so long your mind can get into these creative places, the kind of creative places that come to you when you're halfway between asleep and awake.

For instance, I remember Steve one night saying something about how Atari was planning on using a microprocessor in a game someday soon.

Whoa. I didn't know exactly what a microprocessor was, but I knew enough to know that what we were talking about is a whole little computer inside. And I thought, Wow, a little computer could be inside a game, which either meant that the computer would actually make all the decisions in the game or the game could be a program that used the microprocessor to make it powerful.

I imagined what it might look like someday when microproces-

sors could control games. My brain just took a leap there. There were so many ways it could go.

Then there was another night when some guys were overlaying colored cellophane over the TV screen to make our game look like it had colors in it. As things went from left to right, the colors would sort of shift. And I thought, Oh my gosh, color in computer games would be so neat, that would be just unbelievable!

I used to sit on a bench with Steve on the left-hand side of me breadboarding. And I'd be thinking about how I sort of knew what the waves for color would look like in an oscilloscope. I could imagine it. For instance, one nice pure wave is called a "phase shift." So the way color TV is designed is it has this one particular wave of a certain frequency, a certain number of times per second, which is roughly 3.7593-something cycles per second. Perfect.

According to the theory of phase delay, on an American TV set that particular signal will show up as a color. And there are complicated mathematics and circuits that can introduce the right phase delay to get the color you want. (Also, the signal itself that comes to a TV set can be higher or lower voltage. Higher voltage means lighter—more toward white—while lower voltage is darker—more toward black.)

But somehow this idea popped into my head that if you took a digital chip—a chip that works with 1s and 0s, not waves—and you spun this chip around with four little bits, you end up with four 0s. And four 0s would look just like black on a TV. And what if you put four 1s in? Then you'd get white. Now, say, you put 1, 0, 1, 0, and the rotation was fast enough, it's going to average out gray. So if you could keep spinning this register at the exact right rate, it would come out as the United States color TV frequency, showing up as color on most TV sets. You could even put it through a small filter and round it off the way real color TV set waves work. The concept I came up with is that if I kept shifting

this register around, it just might come out in purple, or red if it was shifted slightly the other way.

How amazing that one little digital chip doing nothing but 1s and 0s could do what color TVs could with waves! It would be so much simpler and more precise.

That was amazing because back then, color TVs operated with circuits a lot more complicated than any computer was back then. And the funny thing is, that very idea came to me in the middle of the night at that lab at Atari. I did no testing on it, but I filed it away in my memory, and eventually that was exactly how things like color monitors ended up on personal computers everywhere. Because of my wild idea that night.

• ○ •

In addition to thinking, while I was waiting for Steve to finish breadboarding I also spent a lot of time playing what I thought was the best game ever: Gran Trak 10. In just those couple of nights, I got so good at it that many years later, when I found one in a pizza parlor, I was able to get the score you needed for a free pizza every time. After I did that twice, the pizza parlor got rid of the machine.

Maybe you're wondering why I didn't use the extra two hours to sleep instead of playing Gran Trak 10, a racing game I loved. It was because at any moment Steve might call me in and say, "Okay, I've got breadboard. Let's test it." And I had to be there for the testing because I was the one who'd understand the circuit I'd designed.

The bottom line of this story is that I actually did finish this project in four days and nights somehow, and it worked.

Steve and I both ended up with mononucleosis. The whole thing used forty-five chips, and Steve paid me half the seven hundred bucks he said they paid him for it. (They were paying us based on how few chips I could do it in.) Later I found out he got paid a bit more for it—like a few thousand dollars—than he said at the time,

but we were kids, you know. He got paid one amount, and told me he got paid another. He wasn't honest with me, and I was hurt. But I didn't make a big deal about it or anything.

Ethics always mattered to me, and I still don't really understand why he would've gotten paid one thing and told me he'd gotten paid another. But, you know, people are different. And in no way do I regret the experience at Atari with Steve Jobs. He was my best friend and I still feel extremely linked with him. I wish him well. And it was a great project that was so fun. Anyway, in the long run of money—Steve and I ended up getting very comfortable money-wise from our work founding Apple just a few years later—it certainly didn't add up to much.

Steve and I were the best of friends for a very, very long time. We had the same goals for a while. They jelled perfectly at forming Apple. But we were always different people, different people right from the start.

You know, it's strange, but right around the time I started working on what later became the Apple I board, this idea popped into my mind about two guys who die on the same day. One guy is really successful, and he's spending all his time running companies, managing them, making sure they are profitable, and making sales goals all the time. And the other guy, all he does is lounge around, doesn't have much money, really likes to tell jokes and follow gadgets and technology and other things he finds interesting in the world, and he just spends his life laughing.

In my head, the guy who'd rather laugh than control things is going to be the one who has the happier life. That's just my opinion. I figure happiness is the most important thing in life, just how much you laugh. The guy whose head kind of floats, he's so happy. That's who I am, who I want to be and have always wanted to be.

And that's why I never let stuff like what happened with

Breakout bother me. Though you can disagree—you can even split from a relationship—you don't have to hold it against the other. You're just different. That's the best way to live life and be happy.

And I figured this all out even before Steve and I started Apple.

Chapter 10

My Big Idea

I can tell you almost to the day when the computer revolution as I see it started, the revolution that today has changed the lives of everyone.

It happened at the very first meeting of a strange, geeky group of people called the Homebrew Computer Club in March 1975. This was a group of people fascinated with technology and the things it could do. Most of these people were young, a few were old, we all looked like engineers; no one was really good-looking. Ha. Well, we're talking about engineers, remember. We were meeting in the garage of an out-of-work engineer named Gordon French.

After my first meeting, I started designing the computer that would later be known as the Apple I. It was that inspiring.

Almost from the beginning, Homebrew had a goal: to bring computer technology within the range of the average person, to make it so people could afford to have a computer and do things with it. That had been my goal, too, for years and years before that. So I felt right at home there.

And eventually Homebrew's goal just expanded and expanded. It wasn't long before we were talking about a world—a possible world—where computers could be owned by anybody, used by

anybody, no matter who you were or how much money you made. We wanted them to be affordable—and we wanted them to change people's lives.

Everyone in the Homebrew Computer Club envisioned computers as a benefit to humanity—a tool that would lead to social justice. We thought low-cost computers would empower people to do things they never could before. Only big companies could afford computers at the time. That meant they could afford to do things smaller companies and regular people couldn't do. And we were out to change all that.

In this, we were revolutionaries. Big companies like IBM and Digital Equipment didn't hear our social message. And they didn't have a clue how powerful a force this small computer vision could be. They looked at what we were doing—small computers, hobby computers—and said they would just remain toys. And a relatively minor business. They didn't imagine how they could evolve.

There was a lot of talk about our being part of a revolution. How people lived and communicated was going to be changed by us, changed forever, changed more than anyone could predict exactly.

Of course there was also a lot of talk about specific components that would make faster computers, and about technical solutions for computers and accessories themselves. People would talk about the humanistic future uses of computers. We thought computers were going to be used for all these weird things—strange geeky things like controlling the lights in your house—and that turned out not to be the case. But everyone felt this thing was coming. A total change. We couldn't always define it, but we believed it.

As I said, almost all of the large computer companies were on record saying that what we were doing was insignificant. It turned out they were wrong and we were right—right all the way.

But back then, even we had no idea how right we were and how huge it would become.

• ○ •

It's funny and maybe a little bit ironic how my involvement in the whole Homebrew thing got started. Remember Allen Baum? He shows up again and again at a lot of important times in my life. He was my friend who sometimes worked at Sylvania with me in high school, whose dad designed the TV Jammer, who did the Homestead High prank with Steve Jobs and me, and also the one who helped get me that dream job at Hewlett-Packard.

I still had that HP job at the time. One day at work I got a call from Allen. It was a call that would change my life yet again, the call that introduced me to Homebrew.

Allen called and said something like, "Listen. There's this flyer I found at HP, it's for a meeting of people who are building TV and video terminals and things."

Now, TV terminals I already knew a little about. By this point, in 1975, I'd done all kinds of side projects, and had already learned a lot about putting data from computers onto TVs. Not

More About Homebrew

This Homebrew Club I belonged to since its first meeting in March 1975 led to other computer companies than Apple. It was incredibly revolutionary. Other members who started computer companies included Bob Marsh and Lee Felsenstein (Processor Technology), Adam Osborne (Osborne Computers), and, of course, me and Steve Jobs, who I later talked into going with me. I once wrote an article on the importance of Homebrew, and you can find it at: http://www.atariarchives.org/deli/homebrew_and_how_the_apple.php.

only had I done my version of Pong plus that project at Atari, Breakout, but I'd already built a terminal that could access the ARPANET, the government-owned network of computers that was the predecessor to the Internet. My terminal even let you display a few letters, up to sixty characters a second. I know that sounds slow now, but this was about six times faster than most teletype systems at the time and a whole lot cheaper. Teletype systems cost thousands of dollars, way more than someone on an engineer's salary could afford, but I built a system using a Sears TV and a cheap $60 typewriter keyboard.

Just like my Pong design and the Cartrivision VCR, I connected my video signal into the test pin of my home TV, the one I found in the schematics.

Now, if Allen had told me that Homebrew was going to be about microprocessors, I probably wouldn't have gone. I know I wouldn't have gone. I was shy and felt that I knew little about the newest developments in computers. By this time, I was so totally out of computers. I was just immersed in my wonderful calculator job at HP. I wasn't even following computers at all. I mean, I hardly even knew what the heck a microprocessor was.

But, like I said, I thought it was going to be a TV terminal meeting. I thought, Yeah, I could go to this thing and have something to say.

I was scared, but I showed up. And you know what? That decision changed everything. That night turned out to be one of the most important nights of my life.

• ○ •

About thirty people showed up for this first meeting there in that garage in Menlo Park. It was cold and kind of sprinkling outside, but they left the garage door open and set up chairs inside. So I'm just sitting there, listening to the big discussion going on.

They were talking about some microprocessor computer kit being up for sale. And they seemed all excited about it. Someone

there was holding up the magazine *Popular Electronics*, which had a picture of a computer on the front of it. It was called the Altair, from a New Mexico company named MITS. You bought the pieces and put them together and then you could have your own computer.

So it turned out all these people were really Altair enthusiasts, not TV terminal people like I thought. And they were throwing around words and terms I'd never heard—talking about microprocessor chips like the Intel 8080, the Intel 8008, the 4004, I didn't even know what these things were. Like I said, I'd been designing calculators for the last three years, so I didn't have a clue.

I felt so out of it—like, No, no, I am not of this world. Under my breath, I am cussing Allen Baum. I don't belong here. And when they went around and everyone introduced themselves, I said, "I'm Steve Wozniak, I work at Hewlett-Packard on calculators and I designed a video terminal." I might have said some other things, but I was so nervous at public speaking that I couldn't even remember what I said afterward. After that, we all signed a sheet of paper where we were supposed to put down our name and what interests and talents we were bringing to the group. (This piece of paper is public now; you might be able to find it online.) The thing I wrote on that paper was, "I have very little free time."

Isn't that funny? These days I'm so busy and people are constantly asking for my autograph and stuff, but back then I was also just as busy: always working on projects, engineering for work and then engineering at home. I don't feel like I've changed much since then, and I guess this proves it, sort of.

Well, anyway, I was scared and not feeling like I belonged, but one very lucky thing happened. A guy started passing out these data sheets—technical specifications—for a microprocessor called the 8008 from a company in Canada. (It was a close copy,

or clone, of Intel's 8008 microprocessor at the time.) I took it home, figuring, Well, at least I'll learn something.

• ○ •

That night, I checked out the microprocessor data sheet and I saw it had an instruction for adding a location in memory to the A register. I thought, Wait a minute. Then it had another instruction you could use for subtracting memory from the A register. Whoa. Well, maybe this doesn't mean anything to you, but I knew exactly what these instructions meant, and it was the most exciting thing to discover ever. Because I could see right away that these were exactly like the instructions I used to design and redesign on paper for all of those minicomputers back in high school and college. I realized that all those minicomputers I'd designed on paper were pretty much just like this one.

Only now all the CPU parts were on one chip, instead of a bunch of chips, and it was a microprocessor. And it had pins that came out, and all you had to do was use those pins to connect things to it, like memory chips.

Then I realized what the Altair was—that computer everyone was so excited about at the meeting. It was exactly like the Cream Soda Computer I'd designed five years before! Almost exactly. The difference was that the Altair had a microprocessor—a CPU on one chip—and mine had a CPU that was on several chips. The other difference was that someone was selling this one—for $379, as I recall. Other than that, there was pretty much no difference. And I designed the Cream Soda five years before I ever laid eyes on an Altair.

It was as if my whole life had been leading up to this point. I'd done my minicomputer redesigns, I'd done data on-screen with Pong and Breakout, and I'd already done a TV terminal. From the Cream Soda Computer and others, I knew how to connect memory and make a working system. I realized that all I needed was this Canadian processor or another processor like it and

some memory chips. Then I'd have the computer I'd always wanted!

Oh my god. I could build my own computer, a computer I could own and design to do any neat things I wanted to do with it for the rest of my life.

I didn't need to spend $400 to get an Altair—which really was just a glorified bunch of chips with a metal frame around it and some lights. That was the same as my take-home salary, I mean, come on. And to make the Altair do anything interesting, I'd have to spend way, way more than that. Probably hundreds, even thousands of dollars. And besides, I'd already been there with the Cream Soda Computer. I was bored with it then. You never go back. You go forward. And now, the Cream Soda Computer could be my jumping-off point.

No way was I going to do that. I decided then and there I had the opportunity to build the complete computer I'd always wanted. I just needed any microprocessor, and I could build an extremely small computer I could write programs on. Programs like games, and the simulation programs I wrote at work. The possibilities went on and on. And I wouldn't have to buy an Altair to do it. I would design it all by myself.

That night, the night of that first meeting, this whole vision of a kind of personal computer just popped into my head. All at once. Just like that.

• ○ •

And it was that very night that I started to sketch out on paper what would later come to be known as the Apple I. It was a quick project, in retrospect. Designing it on paper took a few hours, though it took a few months longer to get the parts and study their data sheets.

I did this project for a lot of reasons. For one thing, it was a project to show the people at Homebrew that it was possible to build a very affordable computer—a real computer you could

program for the price of the Altair—with just a few chips. In that sense, it was a great way to show off my real talent, my talent of coming up with clever designs, designs that were efficient and affordable. By that I mean designs that would use the fewest components possible.

I also designed the Apple I because I wanted to give it away for free to other people. I gave out schematics for building my computer at the next meeting I attended.

This was my way of socializing and getting recognized. I had to build something to show other people. And I wanted the engineers at Homebrew to build computers for themselves, not just assemble glorified processors like the Altair. I wanted them to know they didn't have to depend on an Altair, which had these hard-to-understand lights and switches. Every computer up to this time looked like an airplane cockpit, like the Cream Soda Computer, with switches and lights you had to manipulate and read.

Instead they could do something that worked with a TV and a real keyboard, sort of like a typewriter. A computer like I could imagine.

As I told you before, I had already built a terminal that let you type regular words and sentences to a computer far away, and that computer could send words back to the TV. I just decided to add the computer—my microprocessor with memory—into the same case as that terminal I'd already built.

Why not make the faraway computer this little microprocessor that's right there in the box?

I realized that since you already had a keyboard, you didn't need a front panel. You could type things in and see things on-screen. Because you have the computer, the screen, and the keyboard, too.

So people now say this was a far-out idea—to combine my terminal with a microprocessor—and I guess it would be for other people. But for me, it was the next logical step.

That first Apple computer I designed—even though I hadn't named it an Apple or anything else yet—well, that was just when everything fell into place. And I will tell you one thing. Before the Apple I, all computers had hard-to-read front panels and no screens and keyboards. After Apple I, they all did.

• ○ •

Let me tell you a little about that first computer—what is now called the Apple I—and how I designed it.

First, I started sketching out how I thought it would work on paper. This is the same way I used to design minicomputers on paper in high school and college, though of course they never got built. And the first thing was I had to decide what CPU I would use. I found out that the CPU of the Altair—the Intel 8080—cost almost more than my monthly rent. And a regular person couldn't purchase it in small or single-unit quantities anyway. You had to be a real company and probably fill out all kinds of credit forms for that.

Luckily, though, I'd been talking to my cubicle mates at HP about the Homebrew Club and what I was planning, and Myron Tuttle had an idea. (You remember him: the guy whose plane almost crashed when I was in it.) He told me there was a deal you could get from Motorola if you were an HP employee. He told me that for about $40, I could buy a Motorola 6800 microprocessor and a couple of other chips. I thought, Oh man, that's cheap. So very quickly I knew exactly what processor I would have.

Another thing that happened really early on was I realized—and it was an important realization—that our HP calculators were computers in a real sense. They were as real as the Altair or the Cream Soda Computer or anything else. I mean, a calculator had a processor and memory. But it had something else, too, a feature computers didn't have at the time. When you turned a calculator on, it was ready to go: it had a program in it that started up and then it was ready for you to hit a number. So it booted up automatically and just sat there, waiting for you to tell it to do some-

thing. Say you hit a "5." The processor in the calculator can see that a button is pushed, and it says, Is that a 1? No. A 2? No. A 3, 4 . . . it's a 5. And it displays a 5. The program in a calculator that did that was on three little ROM (read-only memory) chips—chips that hold their information even if you turn the power off.

So I knew I would have to get a ROM chip and build the same kind of program, a program that would let the computer turn on automatically. (An Altair or even my Cream Soda Computer didn't do anything for about half an hour after you set switches so you could put a program in.) With the Apple I, I wanted to make the job of having a program go into memory easier. This meant I needed to write one small program which would run as soon as you turned your computer on. The program would tell the computer how to read the keyboard. It would let you enter data into memory, see what data was in memory, and make the processor run a program at a specific point in memory.

What took about half an hour to load up a program on the Altair, took less than a minute using a keyboard on the Apple I.

What Is ROM?

Read-only memory (ROM) is a term you'll hear a lot in this book. A ROM chip can only be programmed once and keeps its information even if the power is turned off. A ROM chip typically holds programs that are important for a computer to remember. Like what to do when you turn it on, what to display, how to recognize connected devices like keyboards, printers, or monitors. In my Apple I design, I got the idea from the HP calculators (which used two ROM chips) to include ROMs. Then I could write a "monitor" program so the computer could keep track of what keys were being pressed, and so on.

If you wanted to see what was in memory on an Altair, it might take you half an hour of looking at little lights. But on the Apple I, it took all of a second to look at it on your TV screen.

I ended up calling my little program a "monitor" program since that program's main job was going to be to monitor, or watch, what you typed on the keyboard. This was a stepping point—the whole purpose of my computer, after all, was to be able to write programs. Specifically, I wanted it to run FORTRAN, a popular language at the time.

So the idea in my head involved a small program in read-only memory (ROM) instead of a computer front panel of lights and switches. You can input data with a real keyboard and look at your results on a real screen. I could get rid of that front panel entirely, the one that made a computer look like what you'd see in an airplane cockpit.

Every computer before the Apple I had that front panel of switches and lights. Every computer since has had a keyboard and a screen. That's how huge my idea turned out.

• ○ •

My style with projects has always been to spend a lot of time getting ready to build it. Now that I saw my own computer could be a reality, I started collecting information on all the components and chips that might apply to a computer design.

I would drive to work in the morning—sometimes as early as 6:30 a.m.—and there, alone in the early morning, I would quickly read over engineering magazines and chip manuals. I'd study the specifications and timing diagrams of the chips I was interested in, like the $40 Motorola 6800 Myron had told me about. All the while, I'd be preparing the design in my head.

The Motorola 6800 had forty pins—connectors—and I had to know precisely how each one of those forty pins worked. Because I was only doing this part-time, this was a long, slow process. And several weeks passed without any actual construc-

tion happening. Finally I came in one night to draw the design on paper. I had sketched it crudely before. But that night I came in and drew it carefully on my drafting board at Hewlett-Packard.

It was a small step from there to a completely built computer. I just needed the parts.

• ○ •

I started noticing articles saying that a new, superior-sounding microprocessor was going to be introduced soon at a show, WESCON, in San Francisco. It especially caught my attention that this new microprocessor—the 6502 from MOS Technologies in Pennsylvania—would be pin-for-pin compatible with, electrically the same as, the Motorola 6800 I had drafted my design around. That meant I could just pop it in without any redesigning at all.

The next thing I heard was that it was going to be sold over the counter at MOS Technologies' booth at WESCON. The fact that this chip was so easy to get is how it ended up being the microprocessor for the Apple I.

And the best part is they cost half ($20) of what the Motorola chip would have cost me through the HP deal.

WESCON, on June 16–18, 1975, was being held in San Francisco's famous Cow Palace. A bunch of us drove up there and I waited in line in front of MOS Technologies' table, where a guy named Chuck Peddle was peddling the chips.

Right on the spot I bought a few for $20 each, plus a $5 manual.

Now I had all the parts I needed to start constructing the computer.

• ○ •

A couple of days later, at a regular meeting of the Homebrew Computer Club, a number of us excitedly showed the 6502 microprocessors we'd bought. More people in our club now had microprocessors than ever before.

I had no idea what the others were going to do with their 6502s, but I knew what I was going to do with mine.

To actually construct the computer, I gathered my parts together. I did this construction work in my cubicle at HP. On a typical day, I'd go home after work and eat a TV dinner or make spaghetti and then drive the five minutes back to work where I would sign in again and work late into the night. I liked to work on this project at HP, I guess because it was an engineering kind of environment. And when it came time to test or solder, all the equipment was there.

First I looked at my design on draft paper and decided exactly where I would put which chips on a flat board so that wire between chips would be short and neat-looking. In other words, I organized and grouped the parts as they would sit on the board.

The majority of my chips were from my video terminal—the terminal I'd already built to access the ARPANET. In addition, I had the microprocessor, a socket to put another board with random-access memory (RAM) chips on it, and two peripheral interface adapter chips for connecting the 6502 to my terminal.

I used sockets for all my chips because I was nuts about sockets. This traced back to my job at Electroglas, where the soldered chips that went bad weren't easily replaced. I wanted to be able to easily remove bad chips and replace them.

I also had two more sockets that could hold a couple of PROM chips. These programmable read-only memory chips could hold data like a small program and not lose the data when the power was off.

Two of these PROM chips that were available to me in the lab could hold 256 bytes of data—enough for a very tiny program. (Today, many programs are a million times larger than that.) To give you an idea of what a small amount of memory that is, a word processor needs that much for a single sentence today.

I decided that these chips would hold my monitor program, the little program I came up with so that my computer could use a keyboard instead of a front panel.

What Was the ARPANET?

Short for the Advanced Research Projects Agency Network, and developed by the U.S. Department of Defense, the ARPANET was the first operational packet-switching network that could link computers all over the world. It later evolved into what everyone now knows as the global Internet.

The ARPANET and the Internet are based on a type of data communication called "packet switching." A computer can break a piece of information down into packets, which can be sent over different wires independently and then reassembled at the other end. Previously, circuit switching was the dominant method—think of the old telephone systems of the early twentieth century. Every call was assigned a real circuit, and that same circuit was tied up during the length of the call.

The fact that the ARPANET used packet switching instead of circuit switching was a phenomenal advance that made the Internet possible.

• ○ •

Wiring this computer—actually soldering everything together—took one night. The next few nights after that, I had to write the 256-byte little monitor program with pen and paper. I was good at making programs small, but this was a challenge even for me.

This was the first program I ever wrote for the 6502 microprocessor. I wrote it out on paper, which wasn't the normal way even then. The normal way to write a program at the time was to pay for computer usage. You would type into a computer terminal you were paying to use, renting time on a time-share terminal, and that terminal was connected to this big expensive computer somewhere else. That computer would print out a version of your program in 1s and 0s that your microprocessor could understand.

This 1 and 0 program could be entered into RAM or a PROM and run as a program. The hitch was that I couldn't afford to pay for computer time. Luckily, the 6502 manual I had described what 1s and 0s were generated for each instruction, each step of a program. MOS Technologies even provided a pocket-size card you could carry that included all the 1s and 0s for each of the many instructions you needed.

So I wrote my program on the left side of the page in machine language. As an example, I might write down "LDA #44," which means to load data corresponding to 44 (in hexadecimal) into the microprocessor's A register.

On the right side of the page, I would write that instruction in hexadecimal using my card. For example, that instruction would translate into A9 44. The instruction A9 44 stood for 2 bytes of data, which equated to 1s and 0s the computer could understand: 10101001 01000100.

Writing the program this way took about two or three pieces of paper, using every single line.

I was barely able to squeeze what I needed into that tiny 256-byte space, but I did it. I wrote two versions of it: one that let the press of a key interrupt whatever program was running, and the other that only let a program check whether the key was being struck. The second method is called "polling."

During the day, I took my two monitor programs and some PROM chips over to another HP building where they had the equipment to permanently burn the 1s and 0s of both programs into the chips.

But I still couldn't complete—or even test—these chips without memory. I mean computer memory, of course. Computers can't run without memory, the place where they do all their calculations and record-keeping.

The most common type of computer memory at the time was called "static RAM" (SRAM). My Cream Soda Computer, the

Altair, and every other computer at the time used that kind of memory. I borrowed thirty-two SRAM chips—each one could hold 1,024 bits—from Myron Tuttle. Altogether that was 4K bytes, which was 16 times more than the 256 bytes the Altair came with.

I wired up a separate SRAM board with these chips inside their sockets and plugged it into the connector in my board.

With all the chips in place, I was ready to see if my computer worked.

• ○ •

The first step was to apply power. Using the power supplies near my cubicle, I hooked up the power and analyzed signals with an oscilloscope. For about an hour I identified problems that were obviously keeping the microprocessor from working. At one point I had two pins of the microprocessor accidentally shorting each other, rendering both signals useless. At another point one pin bent while I was placing it in its socket.

But I kept going. You see, whenever I solve a problem on an electronic device I'm building, it's like the biggest high ever. And that's what drives me to keep doing it, even though you get frustrated, angry, depressed, and tired doing the same things over and over. Because at some point comes the Eureka moment. You solve it.

And finally I got it, that Eureka moment. My microprocessor was running, and I was well on my way.

But there were still other things to fix. I was able to debug—that is, find errors and correct them—the terminal portion of the computer quickly because I'd already had a lot of experience with my terminal design. I could tell the terminal was working when it put a single cursor on the little 9-inch black-and-white TV I had at HP.

The next step was to debug the 256-byte monitor program on the PROMs. I spent a couple of hours trying to get the interrupt

version of it working, but I kept failing. I couldn't write a new program into the PROMs. To do that, I'd have to go to that other building again, just to burn the program into the chip. I studied the chip's data sheets to see what I did wrong, but to this day I never found it. As any engineer out there reading this knows, interrupts are like that. They're great when they work, but hard to get to work.

Finally I gave up and just popped in the other two PROMs, the ones with the "polling" version of the monitor program. I typed a few keys on the keyboard and I was shocked! The letters were displayed on the screen!

It is so hard to describe this feeling—when you get something working on the first try. It's like getting a putt from forty feet away.

It was still only around 10 p.m.—I checked my watch. For the next couple of hours I practiced typing data into memory, displaying data on-screen to make sure it was really there, even typing in some very short programs in hexadecimal and running them, things like printing random characters on the screen. Simple programs.

I didn't realize it at the time, but that day, Sunday, June 29, 1975, was pivotal. It was the first time in history anyone had typed a character on a keyboard and seen it show up on their own computer's screen right in front of them.

Chapter 11

The Apple I

I was never the kind of person who had the courage to raise his hand during the Homebrew main meeting and say, "Hey, look at this great computer advance I've made." No, I could never have said that in front of a whole garageful of people.

But after the main meeting every other Wednesday, I would set up my stuff on a table and answer questions people asked. Anyone who wanted to was welcome to do this.

I showed the computer that later became known as the Apple I at every meeting after I got it working. I never planned out what I would say beforehand. I just started the demo and let people ask the questions I knew they would, the questions I wanted to answer.

I was so proud of my design—and I so believed in the club's mission to further computing—that I Xeroxed maybe a hundred copies of my complete design (including the monitor program) and gave it to anyone who wanted it. I hoped they'd be able to build their own computers from my design.

I wanted people to see this great design of mine in person. Here was a computer with thirty chips on it. That was shocking to people, having so few chips. It was like the same amount of chips on an Altair, except the Altair couldn't do anything unless you bought a lot of other expensive equipment for it. My com-

puter was inexpensive from the get-go. And the fact that you could use your home TV with it, instead of paying thousands for an expensive teletype, put it in a world of its own.

And I wasn't going to be satisfied just typing 1s and 0s into it. My goal since high school was to have my own computer that I could program on, although I always assumed the language on the computer would be FORTRAN.

The computer I built didn't have a language yet. Back then, in 1975, a young guy named Bill Gates was starting to get a little bit of fame in our circles for writing a BASIC interpreter for the Altair. Our club had a copy of it on paper tape which could be read in with a teletype, taking about thirty minutes to complete. Also, at around the same time a book called *101 Basic Computer Games* came out. I could sniff the air.

That's why I decided BASIC would be the right language to write for the Apple I and its 6502 microprocessor. And I found out none existed for the 6502. That meant that if I wrote a BASIC program for it, mine could be the first. And I might even get famous for it. People would say, Oh, Steve Wozniak, he did the BASIC for the 6502.

Anyway, people who saw my computer could take one look at it and see the future. And it was a one-way door. Once you went through it, you could never go back.

• ○ •

The first time I showed my design, it was with static RAM (SRAM)—the kind of memory that was in my Cream Soda Computer. But the electronics magazines I was reading were talking about a new memory chip, called "dynamic RAM" (DRAM), which would have 4K bits per chip.

The magazines were heralding this as the first time silicon chip memory would be less expensive than magnetic core memory. Up to this point, all the major computers, like the systems from IBM and Data General, still used core memory.

I realized that 4K bytes of DRAM—what I needed as a minimum—would only take eight chips, instead of the thirty-two SRAM chips I had to borrow from Myron. My goal since high school had always been to use as few chips as possible, so this was the way to go.

The biggest difference between SRAM and DRAM is that DRAM has to be refreshed continually or it loses its contents. That means the microprocessor has to electrically refresh roughly 128 different addresses of the DRAM every one two-thousandth of a second to keep it from forgetting its data.

I added DRAM by writing data to the screen—I held the microprocessor clock signal steady, holding transitions off, during a period called the "horizontal refresh."

You know how a TV scans one line at a time on your TV, from top to bottom? It takes about 65 microseconds (millionths of a second) to scan each line on a U.S. TV. Well, it turns out that about 40 of these microseconds are visible and the other 25 microseconds are not. During this 25-microsecond time, the so-called refresh period, I inserted 16 unique addresses to the DRAM. (I got these addresses for free, using the counters of the terminal, which were generating video signals.)

I had selection chips that selected the address to come from the horizontal and vertical counter chips of the terminal during this period. Amazingly, it only took two of these selection chips and maybe another chip or two worth of logic to do the whole thing. So I actually stole some cycles away from the microprocessor to refresh the DRAM.

I would've had no idea how to get a DRAM chip, but luckily, right around this time someone at the club who worked at AMI offered some 4K-bit DRAM chips for sale at a reasonable price. This was before they were even on the market. I see now that someone must've ripped them off from AMI, but I didn't ask any questions.

I bought eight of them from the AMI guy for about $5 each and

modified my design. I added some wires to the memory connector on the Apple I board so it could accommodate either an SRAM or DRAM board. I plugged the new DRAM board in, and it worked the very first time.

• ○ •

I had been showing off this exciting design of mine to Steve Jobs. He'd gone with me to Homebrew a few times, helping me carry in my TV. He kept asking me if I could build a computer that could be used for time-sharing—like the minicomputer a local company called Call Computer used.

The year before, Steve and I had sold my ARPANET terminal to Call Computer in Mountain View, giving them the rights to build and sell it.

"Sure," I said. "Someday." It could be done, I thought, but it was ages off.

Then he asked if I could add a disk for storage someday. I said, again, "Sure. Someday." This all seemed a long way off.

Then, a few days after I got the AMI DRAMs working, Steve called me at work. He asked me if I'd considered using the Intel DRAMs instead of AMI's.

"Oh, Intel's are the best, but I could never afford them," I told him.

Steve said to give him a minute.

He made some calls and by some marketing miracle he was able to score some free DRAMs from Intel—unbelievable considering their price and rarity at the time. Steve is just that sort of person. I mean, he knew how to talk to a sales representative. I could never have done that; I was way too shy.

But he got me Intel DRAM chips. Once I had them, I redesigned around them. And I was so proud because my computer looked smaller yet. I had to add a couple more chips to my computer to make it work with the Intel DRAMs. But the Intel chips were physically so much smaller than the AMI chips.

I have to stop here and explain what the big deal about having a smaller-sized chip is. Remember when I said my goal since high school had always been to have the fewest chips? Well, that isn't the whole story. One time in high school, I was trying to get chips for a computer I'd designed. My dad drove me down to meet an engineer he knew at Fairchild Semiconductor, the company that invented the semiconductor. I told him I'd designed an existing minicomputer two ways. I found out that if I used chips by Signetics (a Fairchild competitor), the computer had fewer chips than if I used Fairchild chips.

The engineer asked me which Signetics chips I'd used.

I told him the make and model number.

He pointed out that the Signetics chips I'd used in the design were much larger in physical size, with many more pins and many more wires to connect, than the equivalent Fairchild chips. That added complexity.

I was stunned. Because he made me realize in an instant that the simpler computer design would really have fewer connections, not simply fewer chips. So my goal changed, from designing for fewer chips to trying to have the smallest board, in square inches, possible.

Usually fewer chips means fewer connections, but not always.

Back to the Intel DRAM design of the Apple I, switching from AMI to Intel DRAM memories meant I could reduce the total size of the board, even though I had to add a couple extra chips to do it.

And looking back, what a great, lucky decision it was to go with Intel's chips. Because that chip design eventually became the standard for all memory chips, even to this day.

• ○ •

By Thanksgiving of 1975, Steve had been to a few of the Homebrew meetings with me. And then he told me he'd noticed something: the people at Homebrew, he said, are taking the

schematics, but they don't have the time or ability to build the computer that's spelled out in the schematics.

He said, "Why don't we build and then sell the printed circuit boards to them?" That way, he said, people could solder all their chips to a printed circuit (PC) board and have a computer in days instead of weeks. Most of the hard work would already be done. His idea was for us to make these preprinted circuit boards for $20 and sell them for $40. People would think it was a great deal because they were getting chips almost free from their companies anyway.

Frankly, I couldn't see how we would earn our money back. I figured we'd have to invest about $1,000 to get a computer company to print the boards. To get that money back, we'd have to sell the board for $40 to fifty people. And I didn't think there were fifty people at Homebrew who'd buy the board. After all, there were only about five hundred members at this point, and most of them were Altair enthusiasts.

But Steve had a good argument. We were in his car and he said—and I can remember him saying this like it was yesterday: "Well, even if we lose our money, we'll have a company. For once in our lives, we'll have a company."

For once in our lives, we'd have a company. That convinced me. And I was excited to think about us like that. To be two best friends starting a company. Wow. I knew right then that I'd do it. How could I not?

Chapter 12

Our Very Own Company

To come up with the $1,000 we thought we'd need to build ready-made printed circuit boards, I sold my HP 65 calculator for $500. The guy who bought it only paid me half, though, and never paid me the rest. I didn't feel too bad because I knew HP's next-generation calculator, the HP 67, was coming out in a month and would cost me only $370 with the employee discount.

And Steve sold his VW van for another few hundred dollars. He figured he could ride around on his bicycle if he had to. That was it. We were in business.

Believe it or not, it was only a couple of weeks later when we came up with a name for the partnership. I remember I was driving Steve back from the airport along Highway 85. Steve was coming back from a visit to Oregon to a place he called an "apple orchard." It was actually some kind of commune.

Steve suggested a name—Apple Computer.

The first comment out of my mouth was, "What about Apple Records?" This was (and still is) the Beatles-owned record label.

We both tried to come up with technical-sounding names that were better, but we couldn't think of any good ones. Apple was so much better, better than any other name we could think of.

Steve didn't think Apple Records would have a problem since it probably was a totally different business. I had no idea.

So Apple it was. Apple it had to be.

• ○ •

Really soon after that, we met with a friend of Steve's who worked at Atari. This guy said he'd be able to design the basic layout of my printed circuit board, based on my original design, for about $600. That was what we needed so we could take it into a manufacturing company that could mass-produce boards.

We also met with another guy from Atari, Ron Wayne, who Steve thought could be a partner. I remember meeting him for the first time and thinking, Wow, this guy is amazing. He could just sit at a typewriter and type out our whole legal partnership agreement like he's a lawyer. He wasn't a lawyer, but he knew all the legal words. He was a fast talker and he seemed so smart. He was one of those people who seemed to have a quick answer for everything. He seemed to know how to do all the things we didn't.

Ron ended up playing a huge role in those very early days at Apple—this was before we had funding, before we'd done much of anything. He was really the third partner, when I think of it. And he did a lot. He wrote and laid out the early operation manual. After all, he could type stuff. And he could draw. He was the one who did the etching of Newton under the Apple tree that was on the computer manual.

Underneath it was a line from a William Wordsworth poem describing Newton. It said: "A mind forever voyaging through strange seas of thought . . . alone."

Eventually Steve, Ron, and I figured out a partnership agreement that started Apple and included all three of us. Steve had 45 percent, I had 45 percent, and Ron got 10 percent. We both trusted him as someone who'd be able to resolve arguments. Ron started working on the paperwork.

• ○ •

Where Did That Weird Quote Come From?

I had to look this one up. It turns out it is from book 3 of *The Prelude* by William Wordsworth. (A Mind Forever Voyaging is also the name of a video game from 1985. Who knew?)

The lines in full read like this:

> The antechapel where the statue stood
> Of Newton with his prism and silent face,
> The marble index of a mind for ever
> Voyaging through strange seas of Thought, alone.

Before the partnership agreement was even inked, I realized something and told Steve. Because I worked at HP, I told him, everything I'd designed during the term of my employment contract belonged to HP.

Whether that upset Steve or not, I couldn't tell. But it didn't matter to me if he was upset about it. I believed it was my duty to tell HP about what I had designed while working for them. That was the right thing and the ethical thing. Plus, I really loved that company and I really did believe this was a product they should do. I knew that a guy named Miles Judd, three levels above me in the company structure, had managed an engineering group at an HP division in Colorado Springs that had developed a desktop computer.

It wasn't like ours at all—it was aimed at scientists and engineers and it was really expensive—but it was programmable in BASIC.

I told my boss, Pete Dickinson, that I had designed an inexpensive desktop computer that could sell for under $800 and would run BASIC. He agreed to set up a meeting so I could talk to Miles.

I remember going into the big conference room to meet Pete, his boss, Ed Heinsen, and Ed's boss, Miles. I made my presentation and showed them my design.

"Okay," Miles said after thinking about it for a couple of minutes. "There's a problem you'll have when you say you have output to a TV. What happens if it doesn't look right on every TV? I mean, is it an RCA TV, a Sears TV, or an HP product that's at fault?"

HP keeps a close eye on quality control, he told me. If HP couldn't control what TV the customer was using, how could it make sure the customer had a good experience? More to the point, the division didn't have the people or money to do a project like mine. So he turned it down.

I was disappointed, but I left it at that. Now I was free to enter into the Apple partnership with Steve and Ron. I kept my job, but after that I was officially moonlighting. Everybody I worked with knew about the computer board we were going to sell.

Over the next few months, Miles would keep coming up to me. He knew about BASIC-programmable computers because of his division out in Colorado, and even though they didn't want my design, he said he was intrigued by the idea of having a machine so cheap that anyone could own one and program it. He kept telling me he'd been losing sleep ever since he heard the idea.

But looking back, I see he was right. How could HP do it? It couldn't. This was nowhere near a complete and finished scientific engineer's product. Everybody saw that smaller, cheaper computers were going to be a coming thing, but HP couldn't justify it as a product. Not yet. Even if they had agreed, I see now that HP would've done it wrong anyway. I mean, when they finally did it in 1979, they did it wrong. That machine went nowhere.

A few weeks after the meeting, the PC board was finished and working. I was so proud of it. I was at HP showing it off to some engineers when the phone rang at the lab bench.

It was Steve.

"Are you sitting down?"

"No," I told him.

"Well, guess what? I've got a $50,000 order."

"What?"

Steve explained that a local computer store owner had seen me at Homebrew and wanted to buy one hundred computers from us. Fully built, for $500 each.

I was shocked, just completely shocked. Fifty thousand dollars was more than twice my annual salary. I never expected this.

It was the first and most astounding success for Apple the company. I will never forget that moment.

• ○ •

Well, I decided I should run the whole thing by HP one more time. I spoke to Pete again. He told me to run it by legal.

The legal department ran it by every single division of HP. That process took about two weeks.

But HP still wasn't interested, and I received a note from HP's legal department saying they claimed no right to my design.

• ○ •

It turned out that a guy named Paul Terrell was starting a new computer store, called the Byte Shop in Mountain View.

As I said, Terrell had seen me demonstrating my computer at Homebrew, and he'd told Steve to "keep in touch," and Steve followed up with him the next day. Steve showed up barefoot at his office the next day, saying, "Hi. I'm keeping in touch."

What Steve didn't know was that Paul was looking for a product just like ours. Terrell wanted to sell a complete computer to his customers, fully assembled. And that had never been done before. Before us, Paul had been buying Altairs or kits like that, and had technicians soldering them together in the back. Every time he got one built, he could sell it. But he thought he had a lot of interest, a lot more potential customers. Steve told him about

the Apple I I'd designed and Paul realized it was a completely built board, so it was a great product for him.

So suddenly, with Terrell's order, I could see that someone else was interested in the Apple I. That was so unexpected and exciting—and so easy. I mean, we already had a little company that was set up to mass-produce our boards down in Santa Clara. Now, all we needed to do was supply the additional parts and they would solder them on.

But how would we get the parts? That would cost money we didn't have. Allen Baum and his dad, Elmer, loaned us $1,200 to buy some of the parts. But we did end up finding a chip distributor (Cramer Electronics) and got the parts on a thirty-day credit. The chip distributor had to call Paul Terrell to see if he was really going to pay us.

The deal Steve worked out with Paul Terrell was he would pay us cash on delivery for the computers. So Paul Terrell was really financing this whole project, it turned out. When he paid us, we were able to pay for the chips.

The distributor gave us the parts, and then they went into a sealed closet at the Santa Clara company that was manufacturing the boards. On the day they were ready for them, the parts came out of the closet, were accounted for and soldered on, and then we had thirty days to pay for them.

Our first batch of boards was finished in January 1976. There were kits like the Altair out there, but nothing like what we were doing. I remember how, waiting for them, I was just the happiest person in the world. I was so happy around this time. I never truly thought we were going to make money with Apple. That was never in my mind. The only thing on my mind was, Wow, now that I've discovered what a microprocessor can do, there are so many places I can take it. I knew that for the rest of my life, I would have a computing tool for myself.

The potential with the Apple I was blowing my mind. I mean,

I'm around video games, and suddenly I realize that my little computer is going to be able to play games. I imagined word-processing software replacing typewriters someday. I was a fast typist, and I could see we were nowhere near where we needed to be for a computer to replace a typewriter yet, though I could imagine it. I imagined how a computer could help me with all my design work at HP. It just blew me away. Every single thing I thought about on the computer was going to be valuable. I could see it so clearly. And that was all I could think about.

After the boards were finished, we rounded up Steve's friend Dan Kottke and Steve's sister, Patty, to plug chips into sockets for $1 a board. Steve would bring us maybe ten or twenty assembled boards at a time from the manufacturer. And there we would sit on a lab bench in the garage of Steve's parents' house at 11161 Crist Avenue. Then I would plug each assembled board into the TV and keyboard we had there and test it to see if it worked.

If it did, I put it in a box. If it didn't, I'd figure what pin hadn't gotten into the socket right or what circuit was shorted. I'd fix the bad ones and put them in the box. After a dozen or two were in the box, Steve would drive them down to Paul Terrell's store and get paid in cash.

These weren't finished computers as you would think of them today. Paul Terrell ended up having to supply monitors, transformers, keyboards, and even the cases to put the computers in. I'm not sure that's what he expected. I think he thought, based on what Steve Jobs told him, that he was getting a fully built computer.

Back then, we didn't have the volume to do plastic. So Paul would put them into wooden cases—often a Polynesian wood called koa—which was a style thing for us.

We had to come up with a retail price for our literature. After all, we weren't going to sell them just to Paul.

We decided to price them at $666.66 each—a price I came up with because I liked repeating digits. (That was $500, plus a 30 percent markup.)

And you know what? Neither of us even knew the number's satanic connections until Steve started getting letters about it. I mean, what? The number of the Beast. Truly, I had no idea. I hadn't seen the movie *The Exorcist*. And the Apple I was no beast to me.

• ○ •

By now, writing the BASIC interpreter was turning out to be the longest, most complicated single project I'd ever do for Apple.

Man, I sniveled at BASIC back then. Compared to FORTRAN, it was a weak, lightweight language. I thought no one would ever use it, for example, to create the kind of sophisticated programs engineers and scientists use. I could just see where things were going. That book I told you about, *101 Basic Computer Games*, meant you could just type in the programs and have these games.

I'd been writing a BASIC interpreter to run on the Apple I, which was based around the MOS 6502 processor. I figured if I wrote this language really fast—if I worked on it day and night and turned my ideas into something that worked within a couple of months—well, then I would get almost famous. People would say that Steve Wozniak wrote the first BASIC for the 6502, just like they knew Bill Gates for writing the BASIC for Altair. I would be the source, and that was kind of exciting.

I had never taken a class on writing a computer language. In my early college years, Allen Baum would Xerox textbooks at MIT, where he went to school, and send those pages to me. I learned a little that way.

So I understood that computer languages had a grammatical syntax, just like any language, and I knew how they were organized.

I didn't know that the BASIC interpreters that existed for different computers, like DEC's and HP's, were different in any way.